



SEMESTER II SESSION 2025/2026

**SECJ3483 - 01 TEKNOLOGI WEB
(WEB TECHNOLOGY)**

**INDIVIDUAL ASSIGNMENT AND DHL DAC 3.0
Web Application & Robotic Process Automation (RPA)**

LECTURER : EN. NORIZAM BIN KATMON

NAME : INTAN SERINA BINTI ANUAR MUS

MATRIC NO. : A23CS0085

SECTION : 1

Table of Contents

1.0 INTRODUCTION AND SCENARIO DESCRIPTION	4
2.0 SYSTEM ARCHITECTURE	
3.0 API AND JSON STRUCTURE	5
3.1 API Structure and Endpoints	6
3.2 JSON Data Structure	6
4.0 RPA WORKFLOW AUTOMATION EXPLANATION	8
4.1 Workflow Overview	8
4.2 Input and Output Components	8
4.3 Main Workflow	10
5.0 USER INTERFACE SCREENSHOTS WITH DESCRIPTION	12
5.1 Login Interface	12
5.2 Admin Interface	12
5.3 Knowledge Base Interface for Admin	
5.4 Create New Article as Admin	13
5.5 User Interface	
5.6 Knowledge Base Interface as User	14
5.7 Create New Article as User	15
5.8 Edit Article as User	15
THE MANUAL : Step By Step Instruction to Run The System	17

1.0 INTRODUCTION AND SCENARIO DESCRIPTION

The growth of e-commerce and cross-border trade has accelerated the demand for the logistics and shipment management system. For example, companies like DHL have the duty to document and keep a real-time tracking of shipment, and reply to customer enquiries as soon as possible to secure efficient delivery operations. However, collecting and documenting shipment records, responding to customer enquiries and automatically generating business reports manually is time-consuming and more prone to human errors. This suggests that introducing an intelligent system to automate these tasks is necessary to boost the logistics operation and provide high-quality services and better business value.

This project focuses on the development of DHL DAC 3.0 Assignment System – a web-based logistics support system that automates shipment documentation, article management, and content management through the integration of Vue.js, REST API, .json and Robotic Process Automation (RPA) using UiPath. The system aims to reduce repetitive manual work while improving the accuracy, speed and consistency of logistics operations.

The scenario I was assigned to is mainly on managing shipment-related articles and reports in a logistics environment, where information such as delivery updates, customs delays, and warehouse issues must be recorded, summarized, categorized, and stored for monitoring purposes. This process is done manually by staff, which is time-consuming and prone to errors such as duplicate entries, incorrect categorization, and delayed updates. To improve efficiency, the proposed system uses a web application with Vue.js framework with AI and RPA integration, where the AI module automatically generates summaries and tags, the REST API manages JSON data exchange, and the UiPath bot automates repetitive tasks such as validation, status updates, and article report generation.

By combining web application development, API integration, artificial intelligence, and RPA automation, the system provides a practical solution for modern logistics challenges while aligning with the digital transformation goals of supply chain management.

Overall, the DHL DAC 3.0 Assignment System provides a centralized, efficient, and scalable platform for managing logistics-related content and automated processes. It enhances productivity, minimizes operational errors, and supports better decision-making through faster data processing and intelligent automation, making it highly relevant to real-world logistics environments.

2.0 SYSTEM ARCHITECTURE

For this assignment, the system is developed based on a full-stack web architecture including frontend layer, backend layer, REST API layer and RPA automation layer to provide efficient shipment article management and processing. The frontend is built with Vue 3 and Vite and is a component-based interface that uses Vue Router for navigation. The frontend communicates with the backend over HTTP and exchanges JSON data using Axios.

The backend is built using Express running on Node.js, which handles all business logic and REST API endpoints. The system provides CRUD operations through endpoints such as GET, POST, PUT and DELETE for managing shipment articles. CORS is enabled to allow smooth communication between the frontend and backend during development.

For data storage, the system uses a file-based persistence approach using a data.json file. The backend reads and writes data directly to this JSON file, instead of relying on temporary in-memory storage. Every time a user creates, updates, or deletes an article, the system automatically synchronizes the changes with data.json. This ensures that all records are retained even after the server restarts, providing a simple but effective form of persistent storage suitable for this system.

In addition to the web application, the system integrates Robotic Process Automation (RPA) using UiPath to automate repetitive backend tasks. The UiPath workflow interacts with the REST API by sending HTTP requests to retrieve article data, validate records, update statuses, and generate automated reports. This automation layer reduces manual workload and ensures faster processing of shipment-related information by executing predefined workflows based on API responses.

In conclusion, the system architecture is designed with a clear separation where the frontend handles user interaction, the backend manages logic and API processing, the JSON file provides persistent data storage, and UiPath acts as an automation layer to improve efficiency and reduce repetitive manual tasks.

3.0 API AND JSON STRUCTURE

The system uses a RESTful API built with Express to manage shipment articles through standard HTTP methods. The API follows a simple CRUD structure, where each endpoint is responsible for creating, retrieving, updating, or deleting data stored in a JSON file (data.json). All responses are returned in JSON format to ensure consistency and easy communication between the frontend and backend.

3.1 API Structure and Endpoints

The system provides a few core endpoints to ensure the system flows and works smoothly. These endpoints allow the frontend to communicate directly with the backend and perform real-time data operations.

- GET/kb retrieves all shipment articles stored in the system. This endpoint reads data from `data.json` and returns it as a JSON array.
- GET/kb/:id retrieves a single article based on its unique ID. The backend searches the JSON file and returns the matching record.
- POST/kb creates a new shipment article. The request body must contain article details in JSON format, which are then saved into `data.json` file.
- PUT/kb/:id updates an existing article, for example, changing its status from “Draft” to “Processed”
- DELETE /kb/:id removes an article based on its unique ID by deleting it from the json file.

3.2 JSON Data Structure

This system stores knowledge base articles in data.json as an array of objects, which each object represents a single article and follows the structured format. It is to ensure that it supports content management, filtering and workflow processing.

JSON object used in the system includes :

- id as the unique identifier generated using a timestamp to show when the article is created
- title is the short title of the knowledge base article
- summary is a short description that summarizes the article content
- steps is an array of step-by-step instructions for structured guidance
- tags is the array of keywords used for filtering and organization of the articles

- status shows the workflow state of the article which they could be labelled as draft, reviewed and published
- createdAt is a timestamp indicating when the article was created

4.0 RPA WORKFLOW AUTOMATION EXPLANATION

The RPA Workflow Automation for this project is implemented using UiPath Studio through the main workflow file, *Main.xaml*, which helps to automate file ingestion, validation, audit tracking, and reporting for the DHL knowledge management process. The purpose of this automation is to reduce manual effort in handling incoming operational files by automatically retrieving files from Google Drive, checking for duplicates, updating processing logs, and sending execution summary reports through email.

4.1 Workflow Overview

The workflow follows a structured sequence-based architecture consisting of four main phases: Initialization, Data Acquisition, Processing, and Output & Reporting. During the initialization phase, the system loads existing audit records from *hash_log.csv* and *process_log.csv* to ensure the bot identifies duplicate files and maintains processing history.

In the data acquisition phase, the bot connects to Google Drive and retrieves the list of available files for processing. Each file is downloaded to a local folder and prepared for validation. During the processing phase, the system generates MD5 hash values for duplicate checking, updates processing statuses such as SUCCESS, SKIPPED – DUPLICATE, or FAILED, and records the results in the audit logs.

Finally, in the output and reporting phase, the workflow updates the CSV files, calculates the total number of successful, skipped, and failed processes, and generates an HTML summary report. This report is automatically sent to the administrator using SMTP email, providing a complete overview of the execution results. This structured workflow ensures efficient automation, strong audit control, and reliable exception handling throughout the entire process.

4.2 Input and Output Components

The RPA workflow uses several important input and output components to ensure smooth automation and proper tracking of all processed files. The main input source is a Google Drive folder that contains uploaded operational files such as support documents, issue reports, and instruction files as seen in Figure 4.2.1.

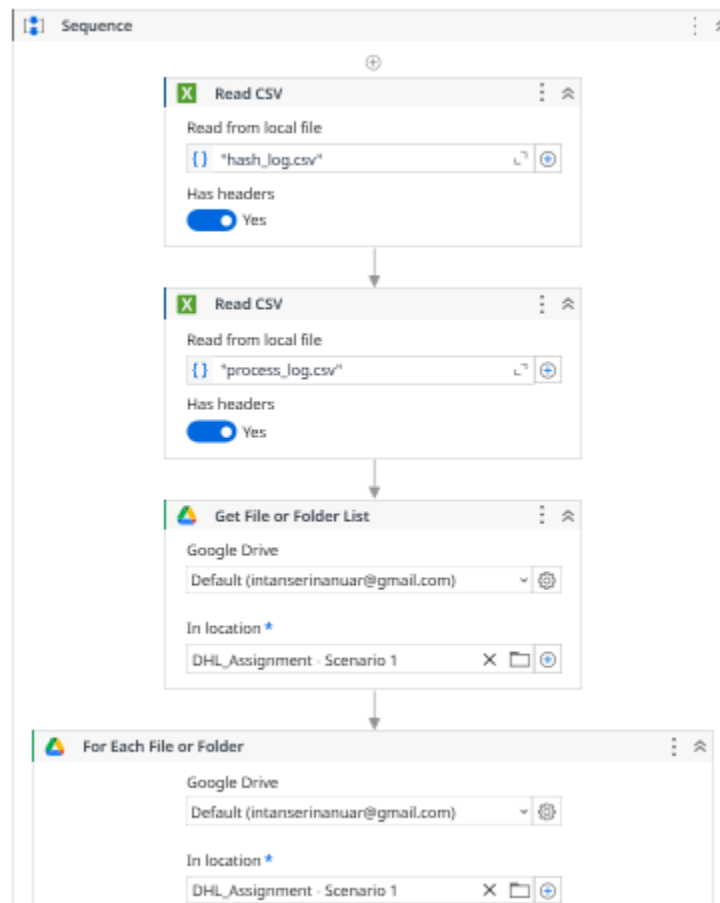
In addition to Google Drive, the workflow also uses two local CSV files, namely *hash_log.csv* and *process_log.csv*, as internal audit records. The *hash_log.csv* file stores MD5 hash values of previously processed files, allowing the bot to detect duplicate files and prevent repeated processing. Meanwhile, the *process_log.csv* file records the status of every file handled by the system, including whether the process was successful, skipped due to duplication, or failed because of an error.

My Drive > DHL_Assignment - Scen... ▾		✓ ☰ ⌵ ⓘ		
Type ▾ People ▾ Modified ▾ Source ▾				
Name ↑	Owner	Last modified ▾	File size	⋮
 Copy of Customer address invalid..txt	 me	11:09 AM me	138 bytes	⋮
 Copy of Customer Credit Approval and New Customer On...	 me	11:09 AM me	22 KB	⋮
 Copy of Error Code AUTH_401.txt	 me	11:09 AM me	155 bytes	⋮
 Copy of Label not printing properly..txt	 me	11:09 AM me	144 bytes	⋮
 Copy of New staff setup steps.txt	 me	11:09 AM me	158 bytes	⋮
 Copy of POD upload failed again.txt	 me	11:09 AM me	175 bytes	⋮
 Copy of RE_ Invoice mismatch issue.msg	 me	11:09 AM me	100 KB	⋮
 Copy of SOP.docx	 me	11:09 AM me	16 KB	⋮
 Copy of Teams Message 2.jpg	 me	11:09 AM me	20 KB	⋮
 Copy of Teams Message.jpg	 me	11:09 AM me	36 KB	⋮

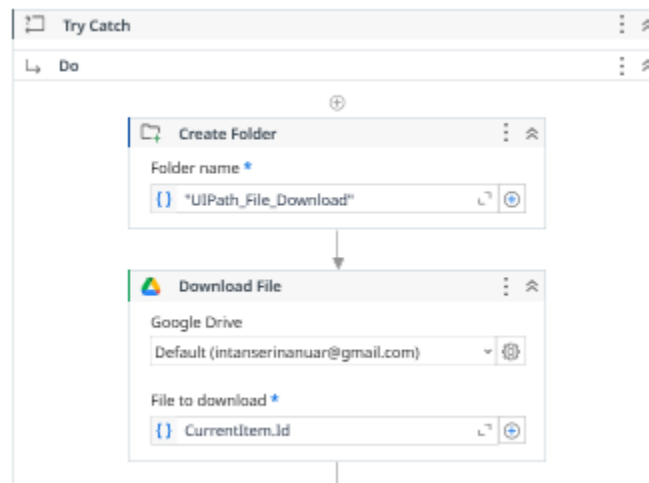
Figure 4.2.1 Uploaded Operational Files from Google Drive

4.3 Main Workflow

The main workflow logic begins by reading the existing audit files, hash_log.csv and process_log.csv, using the Read CSV activity in UiPath. These files are loaded into DataTables named dt_HashLog and dt_ProcessLog, allowing the system to access previous processing records before starting a new execution. This step is important to maintain audit history and prevent duplicate file processing. After loading the audit records, the bot connects to the configured Google Drive folder using the Get File or Folder List activity. All available files are retrieved and stored in the list_DriveFiles variable. The workflow then uses a For Each File or Folder loop to process each file individually. This ensures that every uploaded file is handled systematically without requiring manual selection.



For each file, the bot first creates a local download directory called `UIPath_File_Download` and downloads the file using the **DownloadFileConnections** activity. Once downloaded, the system generates an MD5 hash value for the file and compares it with the existing records in `dt_HashLog`. If the hash already exists, the file is marked as **SKIPPED – DUPLICATE** and no further processing is performed. If the file is new, the workflow continues with validation and processing.



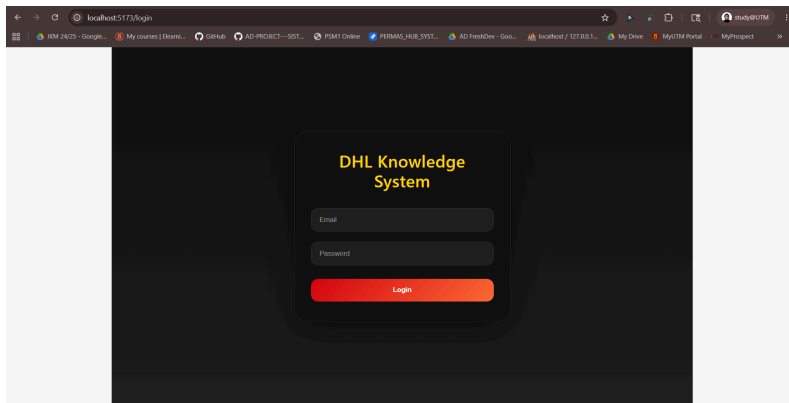
The system then updates the processing status by adding a new row into `dt_ProcessLog` with one of three possible results: SUCCESS, SKIPPED – DUPLICATE, or FAILED. For newly processed files, the corresponding MD5 hash is also added to `dt_HashLog`. This creates a complete audit trail for every file handled by the automation.

To improve reliability, the workflow uses a **Try-Catch** structure during file processing. If an exception occurs, the system automatically captures a screenshot using the **Take Screenshot** activity and records the failure in `process_log.csv`. The workflow then continues processing the next file instead of stopping completely.

After all files have been processed, the workflow writes the updated audit data back into both CSV files using the **AppendWriteCsvFile** activity. It then calculates the total number of successful, skipped, and failed processes using LINQ expressions stored in variables such as `int_SuccessCount`, `int_SkipCount`, and `int_FailCount`. Finally, an HTML summary report is generated and stored in `str_EmailBody`, before being sent automatically to the administrator using the **Send SMTP Email** activity.

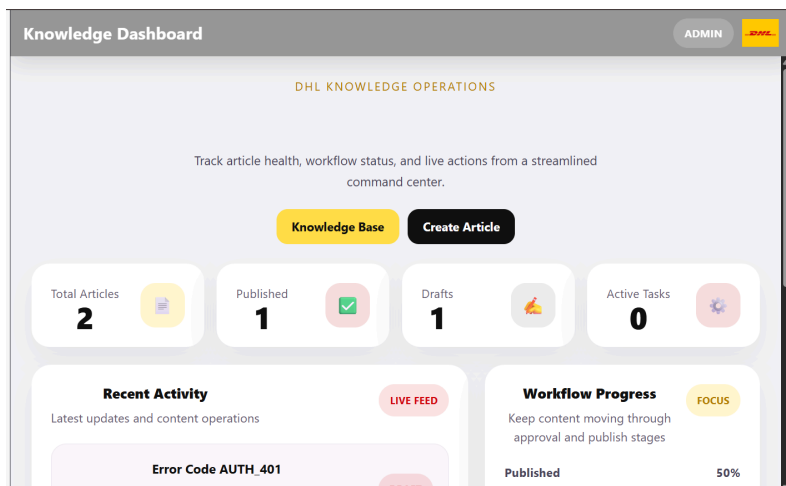
5.0 USER INTERFACE SCREENSHOTS WITH DESCRIPTION

5.1 Login Interface



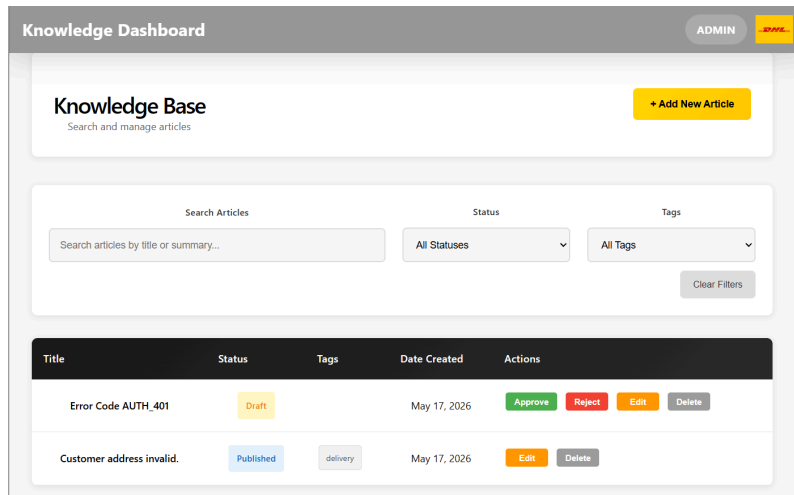
Login interface allows users with roles admin or users to log in with the authenticated credentials admin@dhl.my and user@dhl.my. The administrator could approve or reject an article and write them, while the user could write the article and it could be published only after being approved by the administrator.

5.2 Admin Interface



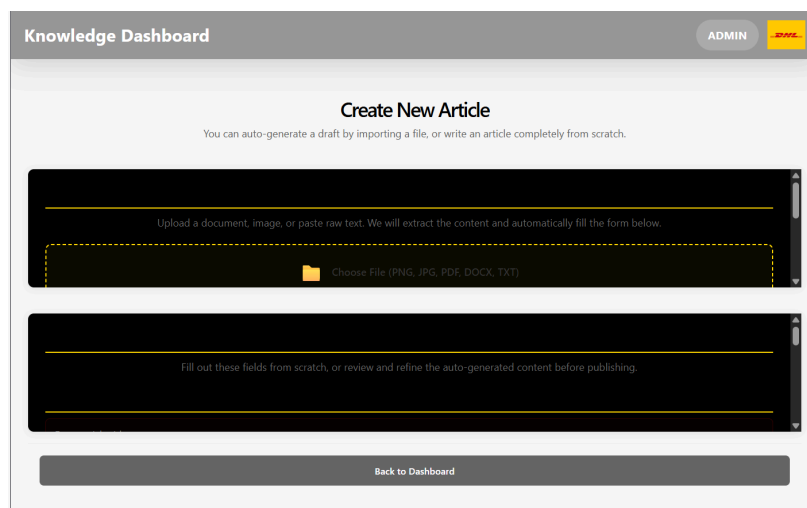
The dashboard for the admin shows the overall statistics with buttons Knowledge Base and Create Article that redirect to their respective pages.

5.3 Knowledge Base Interface for Admin



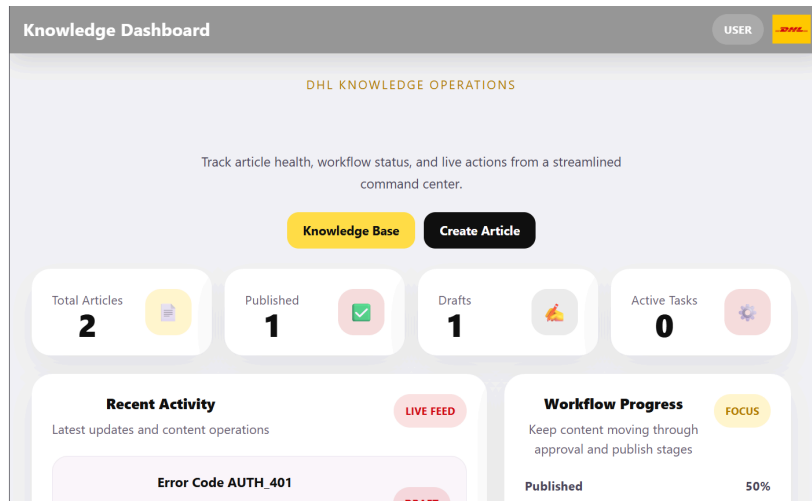
This page lists the articles kept in the database in which as an admin, they could approve or reject or also edit the existing article made by the user.

5.4 Create New Article as Admin



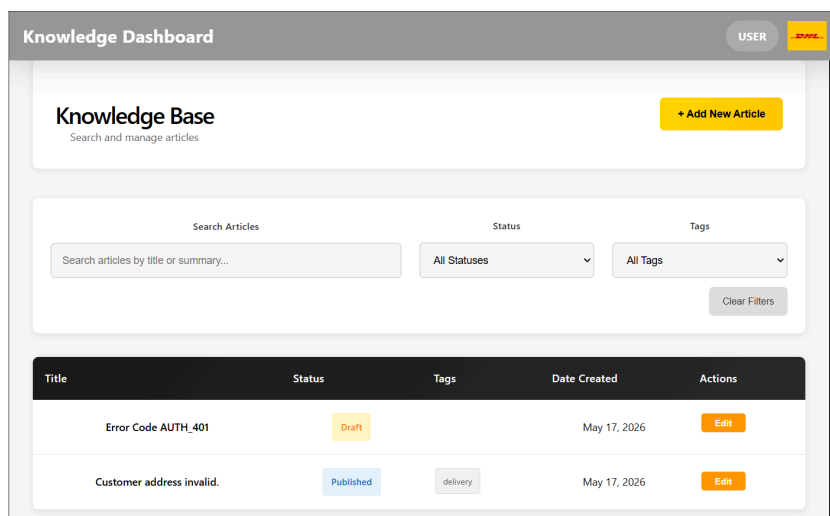
As an admin, they could insert necessary files and enter the details of the article to publish them.

5.5 User Interface



The dashboard for the admin shows the overall statistics with buttons Knowledge Base and Create Article that redirect to their respective pages.

5.6 Knowledge Base Interface as User



This page shows the overall lists of existing articles with their respective statuses, and the user could edit their articles.

5.7 Create New Article as User

The screenshot shows the 'Create New Article' interface within a 'Knowledge Dashboard'. At the top, there's a header with 'Knowledge Dashboard' on the left and 'USER' and a yellow 'Logout' button on the right. Below the header, the title 'Create New Article' is centered, followed by a subtitle: 'You can auto-generate a draft by importing a file, or write an article completely from scratch.' The main content area consists of two large, dark gray rectangular boxes with rounded corners and vertical scrollbars. The first box contains the text 'Upload a document, image, or paste raw text. We will extract the content and automatically fill the the form below.' Below this text is a dashed yellow border representing a file upload area, with a yellow folder icon and the text 'Choose File (PNG, JPG, PDF, DOCX, TXT)'. The second box contains the text 'Fill out these fields from scratch, or review and refine the auto-generated content before publishing.' At the bottom of the form is a gray button labeled 'Back to Dashboard'.

As a user, they could insert necessary files and enter the details of the article to publish them.

5.8 Edit Article as User

The screenshot shows the 'Edit Article' interface within a 'Knowledge Dashboard'. At the top, there's a header with 'Knowledge Dashboard' on the left and 'USER' and a yellow 'Logout' button on the right. Below the header, the title 'Edit Article' is centered, followed by a subtitle: 'Use this page to refine content, save a draft, or publish an article.' The main content area is a light gray form with several sections. The 'Title' section has a text input field containing 'Error Code AUTH_401'. The 'Summary / Content' section has a large text area containing '[Mock Content extracted from Error Code AUTH_401.txt]'. The 'Steps' section contains a list of three steps, each in a yellow box with a red 'X' button: '1 Review auto-generated content', '2 Make necessary edits', and '3 Submit for approval'. Below the steps is a text input field labeled 'Add a new step' and a yellow 'Add Step' button. The 'Tags' section has a text input field containing 'e.g. delivery, tracking, error' and a note 'Separate tags with commas.' At the bottom are two buttons: a red 'Save Changes' button and a gray 'Reset' button.

Users could edit the articles they published by modifying the fields.

5.9 Approve / Reject Article as Admin

Title	Status	Tags	Date Created	Actions
Error Code AUTH_401	Draft		May 17, 2026	Approve Reject Edit Delete
Customer address invalid.	Published	delivery	May 17, 2026	Edit Delete

As an admin, they could approve or reject an article before the user could publish their articles.

THE MANUAL : Step By Step Instruction to Run The System

Step 1 : Install Project Dependencies

To run the DHL Knowledge Management System successfully, users must first install all required dependencies for both the backend and frontend modules. The backend folder should be opened first using a terminal, and the command `npm install` should be executed to install all required Node.js packages such as Express and other supporting libraries. After that, users need to open another terminal for the frontend folder and run `npm install` to install the required frontend packages including Vue 3 and Vite.

First, open a terminal and navigate to the backend folder:

```
PS C:\Users\intan\dhl-assigment-system\backend> npm install axios
```

Next, open another terminal and navigate to the frontend folder:

```
PS C:\Users\intan\dhl-assigment-system\frontend> npm install axios
```

Step 2: Start the Backend API Server

After the installation is completed, the backend API server must be started first. Users need to navigate to the backend folder and run the command `npm start`. This will start the Node.js and Express server, which handles all REST API requests and manages the JSON-based persistent storage file, `data.json`. The backend API runs on `http://localhost:3000`, where all CRUD operations such as creating, retrieving, updating, and deleting knowledge base articles are processed.

```
PS C:\Users\intan\dhl-assigment-system> cd backend && node server.js
KB System running on http://localhost:3000
```

Step 3: Start the Frontend Application

Next, the frontend application must be started by navigating to the frontend folder and running the command `npm run dev`. This starts the Vite development server for the Vue.js web

application. The frontend normally runs on <http://localhost:5173>, which serves as the main user interface where users can manage articles, monitor system activity, and view the dashboard.

```
PS C:\Users\intan\dhl-assigment-system> cd frontend && npm run dev

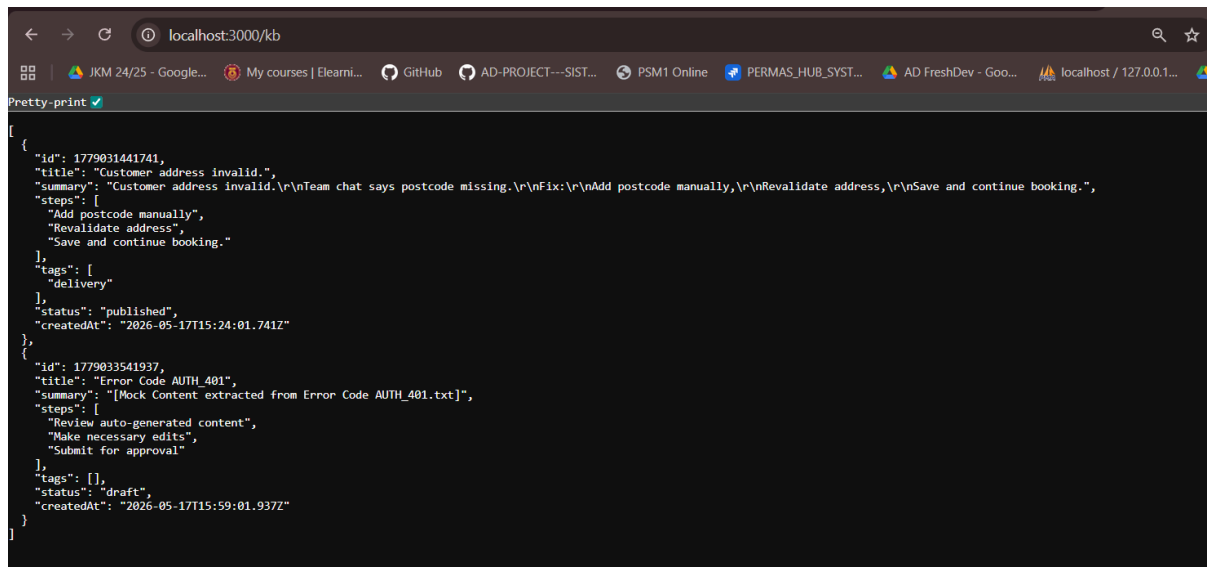
> frontend@0.0.0 dev
> vite

VITE v8.0.12 ready in 792 ms

→ Local:   http://localhost:5173/
→ Network: use --host to expose
→ press h + enter to show help
```

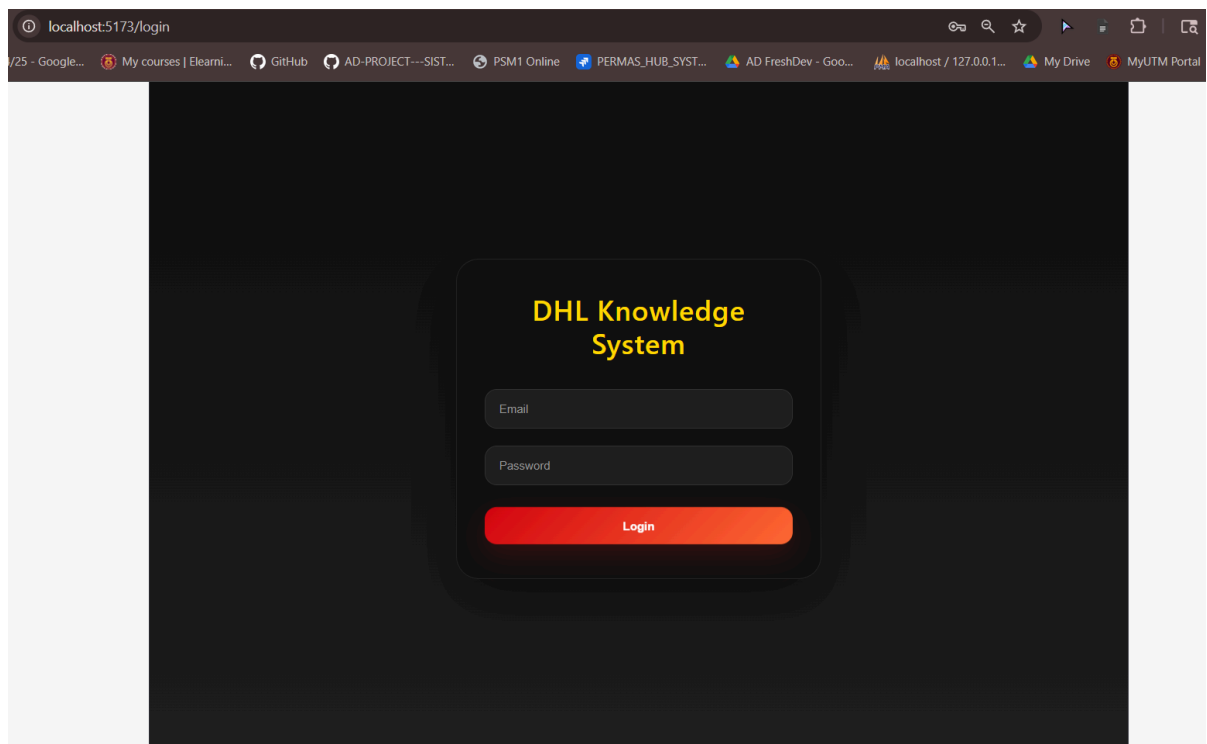
Step 4: Open the Web Application

To verify that the frontend and backend are properly connected, users can open `http://localhost:3000/kb` in the browser. If the backend is functioning correctly, JSON data containing the stored article records will be displayed. Returning to the frontend dashboard at `http://localhost:5173`, users should be able to view the same data presented through the interface. This confirms successful API communication between both system layers.



A screenshot of a web browser window with the address bar showing `localhost:3000/kb`. The browser's developer tools are open, displaying a JSON array of two article objects. The first object has an ID of 1779031441741, a title 'Customer address invalid.', and a summary detailing a team chat about a missing postcode and steps to resolve it. The second object has an ID of 1779033541937, a title 'Error Code AUTH_401', and a summary about mock content extraction. Both objects include tags, status, and creation timestamps.

```
[
  {
    "id": 1779031441741,
    "title": "Customer address invalid.",
    "summary": "Customer address invalid.\nTeam chat says postcode missing.\nFix:\nAdd postcode manually,\nRevalidate address,\nSave and continue booking.",
    "steps": [
      "Add postcode manually",
      "Revalidate address",
      "Save and continue booking."
    ],
    "tags": [
      "delivery"
    ],
    "status": "published",
    "createdAt": "2026-05-17T15:24:01.741Z"
  },
  {
    "id": 1779033541937,
    "title": "Error Code AUTH_401",
    "summary": "[Mock Content extracted from Error Code AUTH_401.txt]",
    "steps": [
      "Review auto-generated content",
      "Make necessary edits",
      "Submit for approval"
    ],
    "tags": [],
    "status": "draft",
    "createdAt": "2026-05-17T15:59:01.937Z"
  }
]
```



Step 6: Run the UiPath RPA Automation

After verifying system connectivity, the UiPath RPA automation can be executed. Users need to open the UiPath project and run the `Main.xaml` workflow. The robot will automatically retrieve files from Google Drive, validate incoming files, update `hash_log.csv` and `process_log.csv`, capture screenshots for failed processes, and generate an execution summary email. If API integration is enabled, the bot will also send processed article data to the backend API endpoint at `http://localhost:3000/api/articles`, allowing new records to appear automatically in the frontend dashboard.

